# Proyecto Práctico: Segundo Examen Parcial

## Flota de Vehículos (p115_SegundoExamenParcial)

**Institución:** Universidad Autónoma de Zacatecas
**Materia:** Programación Orientada a Objetos 1
**Profesor:** Prof. Carlos Héctor Castañeda Ramírez
**Alumno:** Joxan Eliu Solis Rodriguez

## 1. Resumen del Proyecto

El objetivo de este proyecto es implementar un sistema de gestión para una flota de vehículos de transporte en Java, aplicando herencia, clases abstractas, colecciones dinámicas y polimorfismo. Se utilizó un flujo de trabajo asistido de 5 fases iterativas para la Ingeniería de Prompts.

## 2. Fase 1: Análisis y Modelado (UML)

### Prompt Utilizado (Fase 1)

```
Actúa como un arquitecto de software UML. Genera ÚNICAMENTE el código en formato Mermaid para el diagrama de clases del sistema Flota de Vehículos. Representa la clase base abstracta Vehiculo, las subclases CamionCarga y AutobusPasajeros, la clase gestora Flota y sus relaciones. No generes código Java aún.
```

```
+-------------------------------------------------------------+ | <<abstract>> Vehiculo | +---
-------------------------------------------------------------+ | - marca: String | - modelo:
String | - capacidad: int | | - costoBase: double | # costoTotal: double | +-----------------
-----------------------------------------+ | + abstract getBonoConductor(): double | | +
toString(): String | +-------------------------------------------------------------+ ^ |
(extends) +------------------+------------------+ | | +-------------------------+ +-------
-----------------------+ | CamionCarga | | AutobusPasajeros | +-------------------------+ +-
----------------------------+ | - ejes: int | | - rutas: int | | - kilometraje: double | | -
pasajerosTransportados: int| +-------------------------+ +----------------------------+ |
+ getBonoConductor() | | + getBonoConductor() | +-------------------------+ +--------------
---------------+
```

## 3. Fase 2: Scaffolding y Fase 3: Lógica y Polimorfismo

### Prompt de Scaffolding (Fase 2)

```
Genera el esqueleto base en Java para Vehiculo, CamionCarga y AutobusPasajeros
con sus atributos privados, constructores y getters/setters. No implementes lógica de
cálculo ni toString.
```

### Fase 3: Reglas de Negocio Implementadas

Se implementó el método abstracto `getBonoConductor()` con la anotación `@Override`:

- **Camión Carga:** Bono = (CostoBase * 0.10) + (Ejes * 50) + (Kilometraje * 0.5)
- **Autobús Pasajeros:** Bono = (CostoBase * 0.15) + (Rutas * 120) + (Pasajeros * 0.1)

## 4. Fase 4: Integración y Ejecución de Pruebas

### Salida del Programa (Consola)

```
Reporte Inicial Flota [Nombre División Norte, Ciudad Monterrey, Vehículos = 0, BonoTotal = 0.0,
CostoTotal = 0.0, Camiones = 0, Autobuses = 0] >> Reporte Detallado de la Flota <<<<<
CamionCarga [Vehiculo [Marca=Volvo, Modelo=FMX, Capacidad=30000, CostoBase=2500.00], Ejes=4,
Kilometraje=5000.0, Bono=2950.0, CostoTotal=5450.0] CamionCarga [Vehiculo [Marca=Kenworth,
Modelo=T680, Capacidad=28000, CostoBase=1800.00], Ejes=3, Kilometraje=4200.0, Bono=2430.0,
CostoTotal=4230.0] AutobusPasajeros [Vehiculo [Marca=Mercedes, Modelo=O500, Capacidad=50,
CostoBase=1200.00], Rutas=5, PasajerosTransportados=8500, Bono=1630.0, CostoTotal=2830.0]
AutobusPasajeros [Vehiculo [Marca=Irizar, Modelo=i6, Capacidad=40, CostoBase=1500.00], Rutas=8,
PasajerosTransportados=12000, Bono=2385.0, CostoTotal=3885.0] AutobusPasajeros [Vehiculo
[Marca=Scania, Modelo=K360, Capacidad=45, CostoBase=950.00], Rutas=4,
PasajerosTransportados=6200, Bono=1242.5, CostoTotal=2192.5] Flota [Nombre División Norte,
Ciudad Monterrey, Vehículos = 5, BonoTotal = 10637.5, CostoTotal = 18587.5, Camiones = 2,
Autobuses = 3]
```

## 5. Fase 5: Explicación Técnica (Late Binding)

**Explicación de Polimorfismo:** La clase `Flota` administra la colección utilizando el tipo genérico `List<Vehiculo>`. Al iterar con un ciclo for-each e invocar `getBonoConductor()`, la JVM resuelve dinámicamente en tiempo de ejecución (runtime) a qué clase concreta pertenece el objeto en memoria (CamionCarga o AutobusPasajeros) a través del enlace tardío (late binding), ejecutando la fórmula correspondiente.